

```

import pandas as pd
from country_converter import CountryConverter
from pymongo import MongoClient
# from database import MongoRepository

# Tasks 7.4.5, 7.4.6, 7.4.9, 7.4.10
class MongoRepository:
    def __init__(
        self,
        client=MongoClient(host="localhost", port=27017),
        db="wqu-abtest",
        collection="ds-applicants"
    ):

        """init

        Parameters
        -----
        client : pymongo.MongoClient, optional
            By default MongoClient(host="localhost", port=27017)
        db : str, optional
            By default "wqu-abtest"
        collection : str, optional
            By default "ds-applicants"
        """
        self.collection = client[db][collection]

    def get_nationality_value_counts(self, normalize=True):
        """Return nationality value counts.

        Parameters
        -----
        normalize : bool, optional
            Whether to normalize frequency counts, by default True

        Returns
        -----
        pd.DataFrame
            Database results with columns: 'count', 'country_name', 'country_iso2',
            'country_iso3'.
        """
        # Get result from database
        result = self.collection.aggregate([
            {
                "$group": {
                    "_id": "$countryISO2",
                    "count": {"$count": {}}
                }
            }
        ])

        # Store result in DataFrame
        df_nationality = (
            pd.DataFrame(result)
            .rename(columns={"_id": "country_iso2"})
            .sort_values("count")
        )

        # Add country names and ISO3
        cc = CountryConverter()
        df_nationality["country_name"] = cc.convert(
            df_nationality["country_iso2"], to="name_short"
        )

```

```

df_nationality["country_iso3"] = cc.convert(
    df_nationality["country_iso2"], to="ISO3"
)

# Transform frequency count to pct
if normalize:
    df_nationality["count"] = (
        df_nationality["count"] / df_nationality["count"].sum()
    ) * 100

# Return DataFrame
return df_nationality

def get_ages():
    """Gets applicants ages from database.

    Returns
    -----
    pd.Series
    """
    # Get ages from database

    # Load results into series

    # Return ages
    pass

def __ed_sort(self, x):
    """Helper function for self.get_ed_value_counts."""
    degrees = [
        "High School or Baccalaureate",
        "Some College (1-3 years)",
        "Bachelor's degree",
        "Master's degree",
        "Doctorate (e.g. PhD)",
    ]
    mapping = {k:v for v, k in enumerate(degrees)}
    sort_order = [mapping[c] for c in counts]
    return sort_order

def get_ed_value_counts():
    """Gets value counts of applicant education levels.

    Parameters
    -----
    normalize : bool, optional
        Whether or not to return normalized value counts, by default False

    Returns
    -----
    pd.Series
        W/ index sorted by education level
    """
    # Get degree value counts from database
    result = self.collection.aggregate(
        [
            {
                "$group": {
                    "_id": "$highestDegreeEarned",
                    "count": {"$count": {}}
                }
            }
        ]
    )

```

```

# Load result into Series
education = (
    pd.DataFrame(result)
    .rename({"_id": "highest_degree_earned"}, axis="columns")
    .set_index("highest_degree_earned")
    .squeeze()
)

# Sort Series using `self.__ed_sort`
education.sort_index(key=self.__ed_sort, inplace=True)
# Optional: Normalize Series
if normalize:
    education = (education / education.sum()) * 100
# Return Series
return education

def get_no_quiz_per_day(self):
    """Calculates number of no-quiz applicants per day.

    Returns
    -----
    pd.Series
    """
    # Get daily counts from database
    result = self.collection.aggregate(
        [
            {"$match": {"admissionsQuiz": "incomplete"}},
            {
                "$group": {
                    "_id": {"$dateTrunc": {"date": "$createdAt", "unit": "day"}},
                    "count": {"$sum": 1}
                }
            }
        ]
    )
    # Load result into Series
    no_quiz = (
        pd.DataFrame(result)
        .rename(columns={"_id": "date", "count": "new_users"})
        .set_index("date")
        .sort_index()
        .squeeze()
    )
    # Return Series
    return no_quiz

def get_contingency_table(self):
    """After experiment is run, creates crosstab of experimental groups
    by quiz completion.

    Returns
    -----
    pd.DataFrame
        2x2 crosstab
    """
    # Get observations from database
    result = self.collection.find({"inExperiment": True})

    # Load result into DataFrame
    df = pd.DataFrame(result).dropna()

    # Create cross-tab from DataFrame
    data = pd.crosstab(
        index = df["group"],
        columns = df["admissionsQuiz"],

```

```
        normalize = False  
    ).round(3)
```

```
# Return cross-tab  
return data
```